

Outcome-Weighted and Risk-Aware Behavior Cloning for Chinese Standard Mahjong

Zhiyao Xue ¹ and Wenxin Li ^{1,*}

¹ School of Computer Science, Peking University, Beijing 100871, China; zhiyaoxue25@stu.pku.edu.cn (Z.X.)

* Correspondence: lwx@pku.edu.cn

Abstract

Chinese Standard Mahjong requires decision making under partial observability, multi-player interaction, and a structured action space. We present a compute-efficient supervised system trained from four-player self-play records. The system combines outcome-weighted behavior cloning, residual tile-state encoding, recurrent modeling of ordered discards, opponent-risk supervision, hierarchical legal-action prediction, and deterministic inference control. Paired fixed-wall evaluation shows higher average score and fewer deal-ins for the final system, while an inference stress test demonstrates the value of ordered discard history. Calibration substantially reduced probability error, yet the frozen controller scored lower; controller ablations favored aggregate-risk adjustment. In the Sixth International Mahjong AI Competition, the deployed hybrid bot placed ninth among sixteen finalists.

Keywords: Chinese Standard Mahjong; imperfect-information games; behavior cloning; outcome weighting; opponent modeling; risk-aware decision making; paired evaluation

1. Introduction

Chinese Standard Mahjong, also called Official International Mahjong or Mahjong Competition Rules, is a four-player game with hidden hands, stochastic draws, a large structured action space, and a minimum eight-fan requirement for a legal win. These properties make it a useful benchmark for decision making under partial observability [1]. A capable agent must advance its hand, preserve high-value routes, react to calls, and limit losses to three independently evolving opponents.

Recent Mahjong systems demonstrate the potential of both deep reinforcement learning and supervised learning. Suphx combines supervised initialization and large-scale reinforcement learning for Japanese Mahjong [2]. For Chinese Standard Mahjong, Tjong uses hierarchical decisions, a Transformer-based backbone, fan-oriented reward shaping, and reinforcement-learning fine-tuning [3]. The official competition survey also reports that supervised learning became a common and competitive route when strong self-play logs were available [1]. This motivates a practical question: how far can a resource-conscious system progress by improving the supervision signal, temporal representation, and evaluation protocol before investing in large-scale reinforcement learning?

The project began with a reliable residual policy trained by behavior cloning. This baseline represented public tile information as feature planes, decomposed actions into type and tile choices, applied a legal-action mask, and placed 12th among the 16 qualifiers in the tournament round [4]. Its remaining weakness was high-variance decision quality: ordinary behavior cloning copied weak trajectories, and aggregated discard features ob-

Received:

Accepted:

Published:

Copyright: © 2026 by the authors.

Submitted to *Algorithms* for possible open access publication under the terms and conditions of the [Creative Commons Attribution \(CC BY\)](https://creativecommons.org/licenses/by/4.0/) license.

scored the timing and source of each discard. We therefore focused on costly deal-ins and the conversion of promising hands into score.

This paper makes the following contributions.

1. We formulate an *outcome-weighted behavior-cloning* objective that retains decisions from all four players, smoothly reduces the influence of low-return trajectories, and applies a further penalty to the trajectory ending in a deal-in. The positive weight floor preserves defensive examples.
2. We introduce an *opponent-specific temporal defense representation*: a GRU consumes the true table-wide order of the most recent 80 discards, while auxiliary heads estimate tile-level deal-in probability and loss severity separately for each opponent.
3. We implement a *deterministic auxiliary controller* that combines the masked neural policy with hand-efficiency analysis, eight-fan legality checks, public threat cues, and learned opponent risk. Same-checkpoint ablations separate controller effects from checkpoint quality.
4. We provide a reproducible fixed-wall, controlled-seat evaluation pipeline with per-game logs, latency and safety metrics, strict invalid-wall filtering, and wall-clustered paired confidence intervals. The duplicate concept itself is established in Chinese Standard Mahjong evaluation [1,3]; our contribution is its integration into a reproducible model-comparison workflow.
5. We report component-level positive and negative results. Average score and deal-in rate sometimes moved in different directions; aggregate-risk control matched or exceeded opponent-specific control; and post-hoc calibration reduced downstream score. These findings clarify which forms of added complexity improve play.

These results show that structured supervision and temporal modeling can support competitive Mahjong play within a single-GPU training budget.

2. Related Work

2.1. Mahjong as an imperfect-information benchmark

Mahjong combines long-horizon planning with hidden information and multiplayer interaction. Suphx established a major milestone in Japanese Mahjong through deep reinforcement learning, global reward prediction, oracle guidance, and run-time policy adaptation [2]. Differences in rules, scoring, and deployment constraints limit its direct application to Chinese Standard Mahjong.

Lu et al. formalized Official International Mahjong as an AI research environment and summarized the Botzone/IJCAI competition ecosystem, datasets, rules, and duplicate evaluation format [1]. Their analysis supports the use of visible-state feature encodings and paired competition protocols, and it also highlights the diversity of supervised and reinforcement-learning solutions.

Tjong is the closest published system to our problem setting. It separates action selection from tile selection, uses a Transformer-in-Transformer backbone, trains on 519,338 Botzone game logs, and applies reinforcement-learning fine-tuning with a fan-backward mechanism [3]. Our work differs in objective and resource profile: we retain a compact ResNet-GRU policy, use outcome-weighted supervised learning, explicitly model three opponents' temporal risk, and treat reinforcement learning as a future matched-compute extension.

2.2. Behavior cloning and return-weighted imitation

Behavior cloning estimates a policy directly from logged state-action pairs. It is simple and compute-efficient, but sequential prediction violates the independent and identically distributed assumption: small action errors change later state distributions and

can compound over time [5,6]. Interactive dataset aggregation is difficult here because the teacher is represented by a fixed log archive rather than a queryable expert.

Return- or advantage-weighted regression increases the contribution of actions associated with stronger outcomes. Advantage-weighted regression is commonly expressed as a log-likelihood objective weighted by exponentiated estimated advantage [7]. Our simpler formulation uses a bounded sigmoid of terminal score, omits value estimation during preprocessing, and retains a positive floor for losing trajectories. We call it outcome-weighted behavior cloning.

2.3. Structured policies and temporal opponent modeling

Residual networks provide a convenient inductive bias for local and multi-layer tile patterns [8]. Recurrent networks such as the GRU summarize ordered event histories using comparatively few parameters [9]. Hierarchical action heads reduce interference between qualitatively different decisions; Tjong independently demonstrates the value of separating action type and tile selection in Mahjong [3]. Our model combines these ideas with explicitly opponent-indexed risk targets and a legal 235-action output.

3. Problem Formulation

We model one Mahjong hand as a partially observable stochastic game with four players. At decision step t , player i receives an observation o_t^i derived only from its private hand and public table information, and selects an action a_t^i from the legal set $\mathcal{A}(o_t^i)$. The action vocabulary contains pass, win, discard, chow, pong, exposed kong, concealed kong, and added kong. Tile-specific variants yield 235 discrete action indices.

Given a logged dataset $\mathcal{D} = \{(o_n, a_n, m_n, i_n)\}_{n=1}^N$, where m_n identifies the source game and i_n the acting player, conventional behavior cloning minimizes

$$\mathcal{L}_{\text{BC}}(\theta) = -\frac{1}{N} \sum_{n=1}^N \log \pi_{\theta}(a_n | o_n). \quad (1)$$

This objective assigns equal weight to every recorded action despite substantial differences in terminal outcome. We use game outcome to modulate the imitation target.

The primary deployment objective is expected game score under legal play. We additionally track rank, win rate, self-draw rate, deal-in rate, average fan lost on a deal-in, rate of extreme deal-ins, draw-tenpai rate, call rate, latency, and invalid/timeout rate. No single auxiliary metric is assumed to be a sufficient proxy for policy strength.

4. Method

4.1. Data processing and information boundary

The source archive contains 98,209 Chinese Standard Mahjong self-play records generated by the champion bot of the IJCAI 2020 competition. Each record includes the prevailing wind, four initial hands, the sequence of draws and public actions, terminal fan information when applicable, and the four terminal scores.

Preprocessing replays every record through the feature agent used online. Legal-win, fan, and shanten calculations use PyMahjongGB [10]. A training sample is stored whenever more than one legal action exists. For each decision, the pipeline saves the observation tensor, legal-action mask, recorded action, recent discard sequence with relative player identities, and auxiliary targets. Terminal scores construct training labels and weights and are unavailable to the online policy.

We denote the tile-state tensor for player i at step t by $\mathbf{X}_t^i \in \mathbb{R}^{70 \times 4 \times 9}$. The reported experiments populate 66 planes; four reserved score-context planes remain zero. The policy therefore receives no score or rank context.

4.2. Observable tile-state representation

The 4×9 spatial layout provides slots for 34 tile types plus two unused positions. Count-valued information is represented using cumulative binary planes. The 66 active planes are summarized in Table 1.

Table 1. Active observation planes. Four reserved score-context planes remain zero.

Feature group	Planes	Description
Winds	2	Seat wind and prevailing wind.
Private hand	4	Cumulative count planes for the acting player’s hand.
Visibility	8	Shown-tile counts and estimated remaining-tile counts.
Immediate context	5	Last discard and per-player wall counts.
Discard histories	16	Four cumulative tile-count planes for each player’s river.
Open melds	16	Four cumulative tile-count planes for each player’s packs.
Rule flags	2	After-kong draw and last-tile indicators.
Opponent danger	3	Handcrafted danger estimates for the three opponents.
Discard recency	4	Per-player normalized position of each tile’s latest discard.
Hand progress	6	Shanten, effective tiles, seven-pairs progress, and three flush-route planes.
Total active	66	Stored in a 70-plane compatibility tensor with four zero planes.

4.3. ResNet–GRU policy

The tile-state branch begins with a 3×3 convolution to 128 channels, batch normalization, and ReLU, followed by six residual blocks. A 1×1 head reduces the representation to 64 channels; flattening and a fully connected layer produce a 256-dimensional board embedding.

The temporal branch receives the last 80 discards in true global order. Each event contains a tile index (34 tile types plus padding) and the discarding player’s relative identity. Tile embeddings of dimension 16 and player embeddings of dimension 4 are concatenated and processed by a GRU with hidden size 64. Concatenating board and sequence embeddings yields 320 dimensions, which are fused into a 256-dimensional shared representation. The complete architecture is summarized in Figure 1.

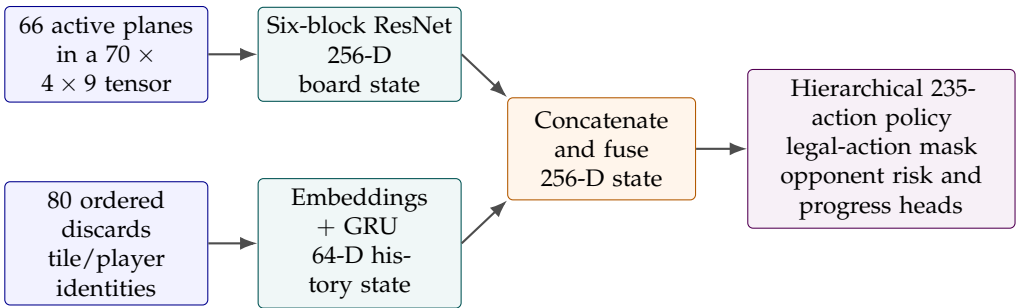


Figure 1. Model overview. The current 70-plane implementation has 2,754,905 trainable parameters; four input planes are zero-padded compatibility channels.

4.4. Hierarchical action representation and legal masking

An eight-way type head predicts Pass, Hu, Play, Chi, Peng, Gang, AnGang, and BuGang. Conditional heads produce logits for 34 discards, 63 chow variants, and 34

variants for each of the four tile-indexed call types. The final 235-action logit for action a is the sum of its type logit and conditional subaction logit. Illegal actions are assigned a logit of -100 before selection.

Training combines the masked full-action cross entropy, a type cross entropy, and conditional subaction losses. The hierarchical factorization shares the trunk while preserving direct competition among all legal actions at inference time.

4.5. Outcome-weighted behavior cloning

Let $S_{m,i}$ denote player i 's terminal score in game m . Let d_m denote the discarder when a hand ends in a discard win and $\emptyset$ otherwise. Let P_m denote the winner's positive terminal settlement score for a winning hand, and set $P_m = 8$ for a draw. We define

$$r_{m,i} = \tanh(S_{m,i}/32), \quad (2)$$

$$g_m = 1 + \log_2\left(\frac{\text{clip}(P_m, 8, 128)}{8}\right), \quad (3)$$

$$w_{m,i} = g_m[0.20 + 0.80 \sigma(3r_{m,i})] \times 0.20^{\mathbb{I}[i=d_m]}. \quad (4)$$

Here, $\mathbb{I}[i = d_m]$ equals one when player i supplies the winning discard and zero otherwise. The bounded sigmoid prevents a few large terminal scores from dominating training, while the 0.20 floor retains losing and defensive trajectories. For a discard win, the final factor downweights the discarding player's entire trajectory in that hand. The outcome-weighted policy loss is

$$\mathcal{L}_{\text{OWBC}}(\theta) = -\frac{1}{N} \sum_{n=1}^N \frac{w_{m_n, i_n}}{\bar{w}} \log \pi_{\theta}(a_n | o_n), \quad (5)$$

where $\bar{w}$ is the minibatch mean weight. This normalization stabilizes the scale of the policy gradient across minibatches with different outcome distributions.

We refer to g_m as a *winner-settlement multiplier*. It uses the winner's terminal score, although each raw record also contains exact fan information. Under Mahjong Competition Rules, discard wins and self-draws apply different settlement transformations to the same fan value. Thus, g_m emphasizes realized return and also reflects win mode. A fan-controlled study would use the terminal fan field directly and retrain all weighting schemes.

4.6. Opponent-specific risk supervision

For each legal discard tile, preprocessing constructs aggregate and opponent-specific binary deal-in targets, together with normalized loss-severity targets. Three structural-tenpai labels are derived from hidden opponent hands during replay. These labels indicate zero shanten; legal waits may still fall below the eight-fan threshold. The model predicts cost-sensitive logits for

$$\hat{p}_j(\text{deal-in} | o, a), \quad \hat{\ell}_j(o, a), \quad \hat{p}_j(\text{tenpai} | o), \quad (6)$$

for relative opponents $j \in \{1, 2, 3\}$. Binary cross entropy uses dynamically clipped positive-class weights to address class imbalance. The resulting sigmoid outputs serve as ranking scores; we assess their probability calibration separately. Loss severity uses smooth L_1 regression.

The auxiliary targets use private replay information during training. At inference, the model estimates hidden danger from public evidence such as melds, visible tiles, and discard timing.

4.7. Auxiliary objectives and deterministic control

Additional heads predict terminal win, a normalized winner-settlement target, shanten, tile-level discard rank, a five-class fan route, and the normalized terminal value of the recorded action. The total training loss is

$$\mathcal{L} = \lambda_p \mathcal{L}_{\text{OWBC}} + \lambda_t \mathcal{L}_{\text{type}} + \lambda_s \mathcal{L}_{\text{sub}} + \sum_k \lambda_k \mathcal{L}_k^{\text{aux}}. \quad (7)$$

Current default coefficients are listed in Table 2. These values were frozen before the fresh-wall confirmatory evaluations.

The competition bot combines the neural checkpoint with a deterministic controller. Let $z_\theta(o_t)$ denote the neural action logits, m_t the legal-action mask, and $h(o_t)$ the controller features. The submitted action is

$$a_t = C_\phi(z_\theta(o_t), m_t, h(o_t)), \quad (8)$$

where C_ϕ has fixed coefficients. During play, it prioritizes a legal win and combines masked policy scores with shanten and effective-tile analysis, eight-fan wait checks, seven-pairs and flush-route cues, call moderation, wall progress, visible melds, discard timing, handcrafted danger, and learned tenpai, deal-in, and severity estimates. A final legality check supplies a deterministic fallback. Every controller input is available from the public game state.

We evaluate three inference configurations. *Raw* applies masked neural argmax. *Light control* adds deterministic legality, hand-progress, and call-management rules. *Risk-aware control* also uses learned risk and tenpai estimates through a deterministic safety gate. The selected deployment configuration disables other experimental auxiliary terms and post-competition logistic calibration; the latter reduced score in paired testing. The supplementary configuration manifest records all controller coefficients. Confirmatory comparisons use wall seeds disjoint from those used to tune the controller and identify whether the subject is a checkpoint or the complete hybrid system.

5. Experimental Setup

5.1. Training configuration

The documented base run used PyTorch 2.8.0 with CUDA 12.8 on one NVIDIA GeForce RTX 5090, BF16 automatic mixed precision, batch size 2048, eight data-loader workers, AdamW [11], and cosine learning-rate decay. The first 90% of records formed the training set and the final 10% the validation set. Starting from the last base checkpoint, outcome-weighted training continued for 12 epochs on samples from all four players. Action-value and opponent-specific severity heads were initialized from scratch, and the optimizer was restarted.

5.2. Comparison design and ablations

Our empirical evaluation examines four aspects. A late-stage checkpoint sweep measures how supervised validation progress translates into game strength. Same-checkpoint comparisons isolate raw argmax, light control, and risk-aware control. A risk-scope ablation compares aggregate and opponent-specific adjustment under a frozen model and controller. Finally, an ordered-sequence stress test replaces discard events with padding to measure temporal reliance. We also compare the final hybrid system with the last base checkpoint and test its sensitivity to four frozen opponents. Checkpoint selection uses validation loss and fixed-wall score alongside policy accuracy. Controlled retraining of winner-only, uniform all-player, and outcome-weighted objectives remains an open causal ablation.

Table 2. Training configuration and loss coefficients.

Item	Value	Item	Value
Games	98,209	Split	90% train / 10% validation
Batch size	2048	Optimizer	AdamW
Initial / minimum LR	10^{-3} / 10^{-5}	Weight decay	10^{-4}
Base / continuation epochs	18 / 12	Precision	BF16
Policy/type/subaction	0.25 / 0.20 / 1.00	Risk aggregate/opponent	0.15 / 0.20
Risk severity agg./opp.	0.10 / 0.10	Opponent tenpai	0.10
Win/fan/shanten	0.05 / 0.10 / 0.05	Discard rank/fan route	0.10 / 0.05
Action value	0.20	Model parameters	2,754,905
Hardware	One RTX 5090	Discard-sequence length	80

5.3. Fixed-wall paired evaluation

For a wall seed w and controlled seat s , candidate c and baseline b face the same three frozen opponents. Their paired score difference is

$$\Delta_{w,s} = R(c;w,s) - R(b;w,s). \tag{9}$$

The primary estimate is the mean paired difference. Since four seat rotations share the same wall, confidence intervals are obtained by nonparametric bootstrap over walls, keeping all four seats in the same cluster. A wall is removed from the strict analysis if any compared policy has an invalid action or timeout on any seat.

The primary protocol uses 1000 fresh, non-overlapping walls per comparison, four controlled seats per wall, and 10,000 wall-cluster bootstrap samples. Any wall containing an invalid action or timeout is excluded in full. Opponent sensitivity is evaluated against four frozen checkpoints spanning different stages of the base training trajectory.

5.4. Metrics and statistical reporting

The primary metric is average score per game. Secondary metrics include average rank, win rate, self-draw rate, deal-in rate, average fan lost on deal-in, extreme deal-in rate (at least 32 fan), draw-tenpai rate, call rate, decision latency, and invalid/timeout rate. Paired effects and 95% confidence intervals provide the main statistical evidence.

Risk prediction is additionally evaluated offline using AUROC, area under the precision-recall curve, Brier score, and expected calibration error (ECE). Platt calibration [12] is fitted on the 90–95% match interval and evaluated on the disjoint 95–100% interval. Results are reported separately for aggregate deal-in risk, flattened opponent-specific risk, and opponent structural tenpai.

6. Results

6.1. Supervised-training dynamics

The base supervised-training stage contains 18 validation points. Policy cross entropy decreased from 0.7745 after the first epoch to a best value of 0.3426 at epoch 17, while policy accuracy increased from 72.75% to 87.09%. Epoch 18 slightly regressed to loss 0.3487 and accuracy 86.93%, supporting checkpoint selection near epoch 17. Mean epoch time was approximately 957 seconds, for about 4.78 hours total, and reported peak CUDA memory was 1.15 GiB. Selected validation points are reported in Table 3.

The same run reported risk precision near 5–6% with recall near 85%, and opponent structural-tenpai precision near 39% with recall near 85%. Class imbalance makes these thresholded metrics insufficient for probability assessment. The disjoint held-out analysis in

Table 8 shows strong ranking performance and severe overconfidence in the weighted-BCE outputs.

Table 3. Selected validation points from the base-stage supervised-training log.

Validation point	Validation loss	Policy accuracy	Epoch time (s)	Peak GPU memory
First recorded	0.7745	0.7275	951.2	1.15 GiB
Best (epoch 17)	0.3426	0.8709	956.5	1.15 GiB
Epoch 18	0.3487	0.8693	957.5	1.15 GiB

6.2. Development checkpoint selection within one training run

The four evaluated checkpoints are late-stage snapshots from one training trajectory. A 500-wall, four-seat sweep against three frozen copies of the epoch-17 checkpoint guided development selection (Table 4). All three later checkpoints improved on epoch 13 by point estimate. The paired difference between epochs 17 and 13 was +1.093 with a 95% wall-clustered CI of [0.230, 1.963]; the intervals for epochs 15 and 18 crossed zero. The sweep identifies epoch 17 as the strongest checkpoint in this development comparison.

Table 4. Sweep over selected late-stage checkpoints from one base training trajectory (500 walls × four controlled seats per checkpoint, light control).

Checkpoint	Avg. score	Avg. rank	Win	Deal-in	Invalid/timeout
After epoch 13	-0.995	2.534	22.60%	17.65%	0.70%
After epoch 15	-0.511	2.518	23.30%	17.50%	0.50%
After epoch 17 (reference)	0.000	2.500	24.00%	16.55%	0.80%
After epoch 18	-0.222	2.514	23.90%	17.60%	0.65%

6.3. Fresh-wall controller and checkpoint comparisons

The same-checkpoint controller ablation used 1000 fresh walls against the reference checkpoint, yielding 12,000 candidate games across three configurations. The uncalibrated risk-aware controller achieved the highest average-score point estimate: 1.458, compared with 1.371 for raw argmax and 1.048 for light control (Table 5). After strict invalid-wall exclusion, the paired difference was +0.168 for risk-aware versus raw (95% CI [−0.220, 0.550]) and −0.230 for light versus raw (95% CI [−0.752, 0.294]). The point estimate therefore selects risk-aware control; both confidence intervals include zero.

On a separate 1000-wall comparison, the final hybrid system raised average score from 0.575 to 1.264, reduced deal-in rate from 18.18% to 15.78%, and improved average rank from 2.499 to 2.451 relative to the last base checkpoint. The paired score difference was +0.677 with 95% CI [−0.139, 1.483]. The gains across score, rank, and deal-in rate show a stronger balance between offense and defense.

6.4. Same-lineage opponent sensitivity

The final hybrid system was also evaluated for 500 walls and four seats against three copies of each locally available base checkpoint. Average score remained positive in all four settings: 2.138, 1.386, 1.788, and 0.918 against checkpoints saved after epochs 13, 15, 17, and 18, respectively (Table 6). Positive scores across the full checkpoint range show that the final policy remains effective as local opponent strength changes.

6.5. Risk scope and temporal-input stress tests

Two post-competition same-checkpoint tests examine how the frozen controller uses auxiliary information (Table 7). Aggregate-risk control achieved a slightly higher average

Table 5. Fresh-wall system comparisons. Each row contains 1000 walls $\times$ four controlled seats. Paired intervals use wall-cluster bootstrap after excluding any wall invalid for either compared system.

Comparison	System	Score	Rank	Win	Deal-in	Paired score Δ (95% CI)
Controller	Final model + raw	1.371	2.458	26.25%	16.90%	Reference
Controller	Final model + light	1.048	2.446	25.20%	15.03%	−0.230 [−0.752, 0.294]
Controller	Final model + risk-aware	1.458	2.443	26.38%	16.10%	+0.168 [−0.220, 0.550]
Checkpoint	Last base + risk-aware	0.575	2.499	24.48%	18.18%	Reference
Checkpoint	Final model + risk-aware	1.264	2.451	25.70%	15.78%	+0.677 [−0.139, 1.483]

Table 6. Final-system sensitivity to four frozen same-lineage opponent checkpoints (500 walls $\times$ four controlled seats per setting).

Frozen opponent	Avg. score	Avg. rank	Win	Deal-in
After epoch 13	2.138	2.436	27.20%	16.30%
After epoch 15	1.386	2.445	25.85%	15.65%
After epoch 17	1.788	2.452	26.60%	16.65%
After epoch 18	0.918	2.478	25.45%	17.25%

score and a slightly lower deal-in rate than opponent-specific control. Among 975 valid paired walls, 904 produced identical mean scores; 32 favored opponent-specific control and 39 favored aggregate control. Aggregate control is therefore the preferred post-competition variant because it is simpler and has better point estimates. The paired difference for opponent-specific versus aggregate risk was -0.137 with 95% CI $[-0.297, 0.016]$; the choice therefore rests on simplicity and point estimates. Opponent-specific targets remain useful during representation learning.

The temporal-input test compared the intact ordered-discard sequence with a configuration in which padding replaced every event at inference. Ordered input increased average score from -0.400 to 2.315 , win rate from 22.15% to 27.28% , and improved average rank from 2.506 to 2.425 . The wall-clustered paired difference was $+2.856$ with 95% CI $[2.115, 3.600]$ over 964 valid walls. Ordered input won on 467 walls, lost on 293, and tied on 204. Deal-in rate changed from 16.13% to 16.40% , indicating broader effects on hand conversion and calls. This large, statistically separated gain demonstrates that ordered discard history is a central component of the trained policy.

Table 7. Fresh-wall risk-scope and ordered-sequence stress tests. Each row contains 1000 walls $\times$ four controlled seats before strict invalid-wall filtering. Paired intervals cluster all four seats by wall.

Test	System	Score	Rank	Win	Deal-in	Paired score Δ (95% CI)
Risk scope	Aggregate risk	2.306	2.423	26.95%	15.33%	Reference
Risk scope	Opponent-specific	2.173	2.431	26.75%	15.75%	−0.137 [−0.297, 0.016]
Temporal input	All-padding sequence	−0.400	2.506	22.15%	16.13%	Reference
Temporal input	Ordered sequence	2.315	2.425	27.28%	16.40%	+2.856 [2.115, 3.600]

6.6. Held-out risk calibration and decision utility

The auxiliary binary heads were trained with clipped positive-class weights, producing cost-sensitive sigmoid scores with poor probability calibration. A positive-slope Platt map was fitted on 293,975 held-out samples from the 90–95% interval and evaluated on 295,427 samples from the disjoint 95–100% interval. Calibration greatly reduced Brier score and 10-bin ECE while preserving monotonic ranking (Table 8 and Figure 2).

Table 8. Disjoint held-out risk evaluation. Arrow columns show raw → Platt-calibrated metrics; AUROC and AUPRC use streaming histogram approximations.

Head	Prevalence	AUROC	AUPRC	Brier score	ECE (10 bins)
Aggregate deal-in	1.892%	0.878	0.133	0.1533 → 0.0173	0.2600 → 0.00033
Opponent deal-in	0.625%	0.889	0.0547	0.0630 → 0.0060	0.1438 → 0.00009
Opponent structural tenpai	16.762%	0.878	0.578	0.1532 → 0.0972	0.1869 → 0.00407

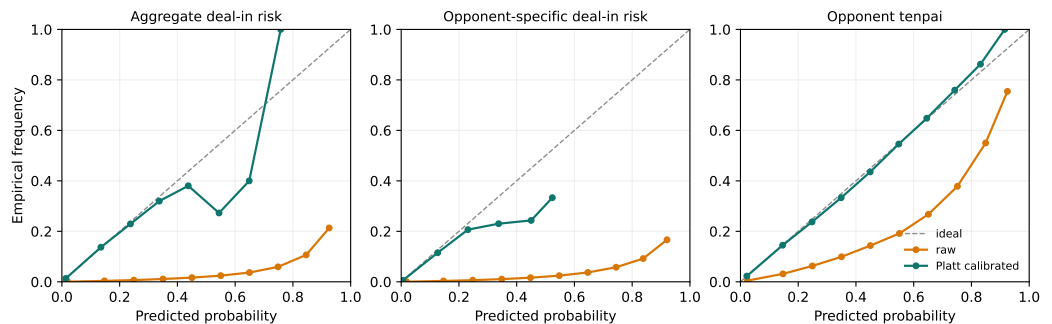


Figure 2. Ten-bin reliability diagrams on the disjoint 95–100% evaluation interval. Raw weighted-BCE outputs are severely overconfident. Platt-calibrated outputs follow the ideal diagonal in well-supported bins; estimates in the sparsely populated high-probability tails remain unstable.

Improved probability estimates reduced the performance of the frozen decision rule. In a 1000-wall paired ablation, the calibrated and uncalibrated configurations shared the checkpoint, controller, opponents, and walls. Calibration reduced deal-in rate from 16.78% to 16.13%, alongside declines in win rate from 25.80% to 25.20% and draw-tenpai rate from 49.13% to 42.16%. Average score fell from 1.502 to 1.253; the paired difference was -0.261 with 95% CI $[-0.509, -0.010]$. The deployment configuration therefore uses the original weighted-BCE scores. Calibrated outputs remain useful for probabilistic interpretation.

6.7. Competition evidence

The deployed hybrid system—the final neural model with its frozen deterministic controller—placed ninth among sixteen finalists, improving on the baseline’s twelfth-place tournament result [4]. The competition followed Mahjong Competition Rules and a Swiss-system duplicate format [4,13]. This result provides an external validation of the complete system under sustained competitive play; the local ablations supply the component-level analysis.

7. Discussion

7.1. Temporal representation and policy quality

The temporal-input experiment confirms that the trained policy uses ordered discard history. Replacing the sequence with padding reduced paired score by 2.856 points per game, with the full confidence interval favoring intact input. Winning and calling also declined, showing that temporal information supports the whole policy. Offline accuracy

alone gave an incomplete picture: the base ResNet–GRU saturated near 87% validation accuracy, yet paired play distinguished checkpoints and inference modes. Fixed-wall evaluation was essential because Mahjong score variance is large relative to policy differences.

Outcome weighting gives every player a usable supervision signal while emphasizing stronger outcomes and suppressing trajectories associated with deal-ins. This design preserves far more training data than winner-only imitation and aligns behavior cloning with game return. The final model’s simultaneous gain in score, rank, and deal-in rate is consistent with this objective. A controlled retraining study can further separate its contribution from continuation training and the new auxiliary heads.

The risk ablations clarify the meaning of defense. Excessive folding can lower deal-in frequency while sacrificing winning opportunities; controlled aggression can raise expected return when losses remain modest. Aggregate-risk control combines simpler inference with the better score and deal-in point estimates, while opponent-specific supervision remains useful as a representation-learning signal. Score, deal-in frequency, and severity together provide a more complete view of defensive quality.

The competition bot benefits from a clear division of labor. The neural policy supplies broad state-dependent action preferences, and the controller contributes domain knowledge together with a transparent legality and safety layer. Risk-aware control achieves the highest score point estimate in the fresh-wall ablation, supporting its selection for deployment.

7.2. Choice of supervised learning

Strong Mahjong systems often employ reinforcement learning [2,3], and RL remains an important next step. It requires a high-throughput simulator, stable self-play population management, careful reward design, and extensive environment interaction. Our supervised pipeline completed 18 epochs in about 4.78 GPU hours on one RTX 5090 and produced a deployable policy directly from fixed logs.

The repository also contains an experimental RL implementation and checkpoints. The next useful comparison should initialize RL from the supervised policy and match either GPU hours or environment steps. This design would measure whether self-play improvement justifies its additional simulation and optimization cost.

7.3. Limitations and threats to validity

The evaluation uses one training lineage and locally available, same-lineage opponents. Wall bootstrap intervals measure variation across walls, while independent training runs and external opponents would extend the evidence to seed variability and heterogeneous playing styles. Differences in data, rules, compute, and protocols also prevent a matched comparison with published systems.

The final model combines outcome weighting, continuation training, and new auxiliary heads. Their individual causal contributions require controlled retraining. Likewise, sequence padding measures the trained policy’s reliance on temporal input and introduces a distribution shift; a separately trained sequence-free model would provide a cleaner architectural comparison.

Risk calibration is reported over the full held-out interval and could be further stratified by hand phase. Controller coefficients and calibrated probabilities require joint tuning. Finally, full public reproducibility requires testing in a clean environment and explicit verification of dataset licensing and redistribution terms.

7.4. Future work

The highest-priority experiments are controlled supervision and representation ablations: winner-only, uniform-all-player, and outcome-weighted behavior cloning; and retrained ordered-GRU, sequence-free, and sequence-shuffled models. A smaller offline ex-

tension could stratify risk calibration by opponent and hand phase. Additional compute or external bot access would also enable multiple training seeds and genuinely heterogeneous opponents.

The calibrated-controller result motivates a joint decision model based on calibrated probability and expected loss, with coefficients retuned on development walls. Reinforcement learning can then serve as a targeted policy-improvement stage initialized from the supervised policy. Candidate approaches include PPO and offline-to-online actor-critic training with a KL constraint to limit policy drift. A fair comparison should match compute, report environment steps and wall-clock cost, and use the same paired evaluation suite.

Other extensions include latent opponent-state modeling, uncertainty-aware risk estimates, and longer-horizon fan-route planning. Fresh-wall score, latency, and legality should guide their selection.

8. Conclusions

We presented a compute-efficient Chinese Standard Mahjong system that combines outcome-weighted behavior cloning, residual tile encoding, ordered-discard modeling, hierarchical legal actions, opponent-risk supervision, and deterministic control. The final system improves the point estimates for score, rank, and deal-in rate over the last base checkpoint. Ordered discard history produces the clearest component gain, with a +2.856 paired score difference and a confidence interval fully above zero. The controller studies further show that aggregate-risk control matched opponent-specific control with a simpler inference rule and that calibrated probabilities require joint retuning with downstream decision thresholds. The complete hybrid bot advanced from twelfth place in the Tournament Round to ninth in Stage One of the Final Round. These results demonstrate that carefully designed supervision, temporal representation, and controlled inference can produce a competitive Mahjong agent within a modest compute budget.

Supplementary Materials

The public reproducibility bundle contains checkpoint-bound source snapshots, manifests, integrity metadata, the base-stage training log, evaluation and bootstrap code, per-game CSV files, aggregate summaries, calibration metrics, and reliability plots. Git excludes raw game records, generated training arrays, model checkpoints, transient logs, and smoke-test artifacts. A versioned archival release will contain model artifacts permitted for distribution. The continuation-stage log and explicit random seed were not preserved, and across-seed variance is unavailable.

Author Contributions: Conceptualization, Z.X.; methodology, Z.X.; software, Z.X.; validation, Z.X.; formal analysis, Z.X.; investigation, Z.X.; data curation, Z.X.; writing—original draft preparation, Z.X.; writing—review and editing, Z.X. and W.L.; visualization, Z.X.; supervision, W.L.; project administration, W.L. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Institutional Review Board Statement: Not applicable. This study analyzes game-state logs and does not involve an intervention with human participants or the collection of directly identifiable personal data.

Informed Consent Statement: Not applicable.

Data Availability Statement: The study uses 98,209 self-play records generated by the champion bot of the IJCAI 2020 competition. Redistribution of the raw archive awaits **[TODO: confirmation of the official access route, release identifier, license, and provider permission]**. Source code and reproducibility materials are available at <https://github.com/Amy22707/Chinese-Standard->

Mahjong. Derived evaluation logs, manifests, and permitted model artifacts will be archived on Zenodo; **[TODO: insert the Zenodo DOI after deposit]**.

Acknowledgments: The authors thank the organizers of the Sixth International Mahjong AI Competition and the Botzone team for providing the competition and evaluation infrastructure. OpenAI Codex was used during manuscript preparation for LaTeX organization, consistency checking, and English-language editing.

Conflicts of Interest: The authors declare no conflict of interest.

1. Lu, Y.; Li, W.; Li, W. Official International Mahjong: A New Playground for AI Research. *Algorithms* **2023**, *16*, 235. <https://doi.org/10.3390/a16050235>.
2. Li, J.; Koyamada, S.; Ye, Q.; Liu, G.; Wang, C.; Yang, R.; Zhao, L.; Qin, T.; Liu, T.-Y.; Hon, H.-W. Suphx: Mastering Mahjong with Deep Reinforcement Learning. *arXiv* **2020**, arXiv:2003.13590. <https://doi.org/10.48550/arXiv.2003.13590>.
3. Li, X.; Liu, B.; Wei, Z.; Wang, Z.; Wu, L. Tjong: A Transformer-Based Mahjong AI via Hierarchical Decision-Making and Fan Backward. *CAAI Transactions on Intelligence Technology* **2024**, *9*, 982–995. <https://doi.org/10.1049/cit2.12298>.
4. Botzone. The Sixth International Mahjong AI Competition: Rules, Timeline, and Results. <https://botzone.org.cn/static/gamecontest2026a.html> (accessed on 28 August 2026).
5. Ross, S.; Bagnell, D. Efficient Reductions for Imitation Learning. In *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics*; PMLR: Chia Laguna Resort, Sardinia, Italy, 2010; Volume 9, pp. 661–668. <https://proceedings.mlr.press/v9/ross10a.html>.
6. Ross, S.; Gordon, G.; Bagnell, D. A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning. In *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics*; PMLR: Fort Lauderdale, FL, USA, 2011; Volume 15, pp. 627–635. <https://proceedings.mlr.press/v15/ross11a.html>.
7. Peng, X.B.; Kumar, A.; Zhang, G.; Levine, S. Advantage-Weighted Regression: Simple and Scalable Off-Policy Reinforcement Learning. *arXiv* **2019**, arXiv:1910.00177. <https://doi.org/10.48550/arXiv.1910.00177>.
8. He, K.; Zhang, X.; Ren, S.; Sun, J. Deep Residual Learning for Image Recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, USA, 27–30 June 2016; pp. 770–778. https://openaccess.thecvf.com/content_cvpr_2016/html/He_Deep_Residual_Learning_CVPR_2016_paper.html.
9. Cho, K.; van Merriënboer, B.; Gulcehre, C.; Bahdanau, D.; Bougares, F.; Schwenk, H.; Bengio, Y. Learning Phrase Representations Using RNN Encoder–Decoder for Statistical Machine Translation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing*, Doha, Qatar, 25–29 October 2014; Association for Computational Linguistics: Stroudsburg, PA, USA, 2014; pp. 1724–1734. <https://doi.org/10.3115/v1/D14-1179>.
10. Peking University AI Lab. PyMahjongGB: Python Fan Calculator for Chinese Standard Mahjong. <https://github.com/ailab-pku/PyMahjongGB> (accessed on 24 August 2026).
11. Loshchilov, I.; Hutter, F. Decoupled Weight Decay Regularization. In *Proceedings of the 7th International Conference on Learning Representations (ICLR)*, New Orleans, LA, USA, 6–9 May 2019. <https://openreview.net/forum?id=Bkg6RiCqY7>.
12. Platt, J.C. Probabilities for SV Machines. In *Advances in Large-Margin Classifiers*; Smola, A.J.; Bartlett, P.L.; Schölkopf, B.; Schuurmans, D., Eds.; MIT Press: Cambridge, MA, USA, 2000; pp. 61–74. <https://doi.org/10.7551/mitpress/1113.003.0008>.
13. International Joint Conferences on Artificial Intelligence. IJCAI 2026 Competitions. <https://2026.ijcai.org/competitions/> (accessed on 28 August 2026).